

BAN404 eksamen-cheatsheet

Ryddig Python-kopi, justert mot forelesers metode og eksamen 2024/2025

2026-05-18

0. Eksamensstrategi

Det som faktisk går igjen.

- Forklar hva kode gjør, linje for linje nok til at metode + objektiv er tydelig.
- Velg data riktig: modellvalg på treningsdata eller kryssvalidering, endelig prediksjon på testdata. For rene “hvorfor”-spørsmål kan full data forsvares, men forklar.
- Deskriptiv statistikk først: riktig figur/tabell for variabeltypene, og skriv konkrete funn med tall.
- Klassifikasjon: ikke bare accuracy. Bruk krysstabell, radproporsjoner og terskel når klassene er ubalanserte.
- Prediksjonscase: fjern variabler som ikke er kjent når prediksjonen skal gjøres, eller som lekker fasiten.
- Vær pragmatisk: en enkel modell som svarer på oppgaven er bedre enn en perfekt modell du ikke rekker.

Imports du nesten alltid trenger.

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf
import sklearn.linear_model as skl
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.preprocessing import StandardScaler
from ISLP.models import ModelSpec as MS, summarize
```

1. Data, oppdeling og evaluering

Forelesers 50/50-split.

```
np.random.seed(123)
n = len(df)
ntrain = n // 2
ind = np.random.choice(np.arange(n), size=ntrain, replace=False)
mask = np.ones(n, dtype=bool)
mask[ind] = False
```

```
train = df.iloc[ind]
test = df.iloc[mask]
```

Ikke bruk `~ind` når `ind` er heltallsindekser. Bruk `mask`, eller:
`test_idx = np.setdiff1d(np.arange(n), ind)`

Forklaringsvariabler og responsvariabel. `X` er forklaringsvariablene/prediktorene. `y` er responsvariabelen vi skal forklare eller predikere.

```
X = pd.get_dummies(df.drop(columns="y"), drop_first=True)
X = X.astype(float).fillna(0)
y = (df["Direction"] == "Up").astype(int)
```

Metrikker. Mean squared error er gjennomsnittlig kvadrert feil. Mean absolute deviation er gjennomsnittlig absoluttfeil. Residual sum of squares er summen av kvadrerte feil. Total sum of squares er total variasjon i responsvariabelen.

```
mse = np.mean((y - pred)**2)
mad = np.mean(np.abs(y - pred)) # foreleser bruker ofte
↳ absoluttfeil
rss = np.sum((y - pred)**2)
tss = np.sum((y - y.mean())**2)
r2 = 1 - rss / tss
accuracy = np.mean(y_true == pred)
```

Krysstabell for klassifikasjon. Brukes for å se riktige og gale klassifikasjoner.

```
cm = pd.crosstab(y_true.values, pred,
                 rownames=["Actual"], colnames=["Predicted"])
row_props = pd.crosstab(y_true, pred, normalize="index")
```

Hvis rader er faktisk og kolonner er predikert:

	Pred 0	Pred 1
Actual 0	TN	FP
Actual 1	FN	TP

Sensitivitet for klasse 1 = $TP / (TP + FN)$. Spesifisitet for klasse 0 = $TN / (TN + FP)$.

2. Deskriptiv statistikk

Velg etter variabeltyper.

- Kontinuerlig `Y`, kontinuerlig `X`: scatter.
- Kontinuerlig `Y`, kategorisk `X`: boxplot.
- Kategorisk `Y`, kontinuerlig `X`: boxplot av `X` etter `Y`.
- Kategorisk `Y`, kategorisk `X`: krysstabell, gjerne `normalize="index"`.

```
df.describe(include="all")
df["y"].value_counts(normalize=True)
pd.crosstab(df["cat"], df["y"], normalize="index")
df.boxplot("x", by="y")
plt.scatter(df["x"], df["y"])
```

Eksamensformulering. “Jeg bruker treningsdata til deskriptiv analyse fordi senere modeller skal sammenlignes på samme testsett” eller “Jeg bruker full data fordi spørsmålet er forklarende/deskriptivt, ikke prediksjon”.

3. Ordinær lineær regresjon og statsmodels

Ordinær lineær regresjon. Minimerer summen av kvadrerte residualer: $\sum (y - \hat{y})^2$.

```
Xc = sm.add_constant(X_train)
ols = sm.OLS(y_train, Xc).fit()
print(ols.summary())
pred = ols.predict(sm.add_constant(X_test))
```

ISLP ModelSpec, forelesers lab-stil. Dette lager designmatrisen som brukes i regresjonen.

```
design = MS(["horsepower"]).fit(Auto)
X = design.transform(Auto)
y = Auto["mpg"]
res = sm.OLS(y, X).fit()
summarize(res)
```

Tolkning.

- Lav `p`-verdi: prediktoren er assosiert med `Y` gitt de andre variablene.
- Konfidensintervall: usikkerhet om forventet respons.
- Prediksjonsintervall: usikkerhet om ny observasjon, derfor bredere.

4. Ridge, LASSO og kryssvalidering

Ridge-regresjon. $\text{sum}((y - \hat{y})^2) + \alpha * \text{sum}(\beta_j^2)$. Krymper koeffisienter, setter dem normalt ikke til null.

LASSO-regresjon. LASSO står for “least absolute shrinkage and selection operator”. Objektivet er $\text{sum}((y - \hat{y})^2) + \alpha * \text{sum}(\text{abs}(\beta_j))$. Kan sette koeffisienter lik null, altså variabelseleksjon.

Bias-variance. Høy α gir enklere modell, høyere bias og lavere varians. Lav α gir mer fleksibel modell, lavere bias og høyere varians.

Sklearn slik foreleser gjør i ex611.py.

```
alphas = np.logspace(-4, 4, 200)
```

```
ridge_cv = skl.RidgeCV(alphas=alphas)
ridge_cv.fit(Xtrain, ytrain)
ridgemin = skl.Ridge(alpha=ridge_cv.alpha_)
ridgemin.fit(Xtrain, ytrain)
pred_ridge = ridgemin.predict(Xtest)
```

```
lasso_cv = skl.LassoCV(alphas=alphas)
lasso_cv.fit(Xtrain, ytrain)
lasso = skl.Lasso(alpha=lasso_cv.alpha_)
lasso.fit(Xtrain, ytrain)
pred_lasso = lasso.predict(Xtest)
```

Koeffisientsammenligning. Kolonnen Linear under er vanlig lineær regresjon.

```
ols_coef = ols.params
ridge_coef = np.concatenate([ridgemin.intercept_, ridgemin.coef_])
lasso_coef = np.concatenate([lasso.intercept_, lasso.coef_])
pd.DataFrame({"Linear": ols_coef, "Ridge": ridge_coef, "Lasso":
    ↪ lasso_coef})
```

Standardisering når skala betyr noe. Gjør variabler sammenlignbare ved å trekke fra gjennomsnitt og dele på standardavvik.

```
scaler = StandardScaler(with_mean=True, with_std=True)
X_train_s = scaler.fit_transform(X_train) # tilpass kun på
    ↪ treningsdata
X_test_s = scaler.transform(X_test)
```

5. Ridge fra scratch, eksamen 2025

Kodefunksjonen i 2025 er Ridge-regresjon. Den trekker fra gjennomsnittet i y og hver kolonne i X , men standardiserer ikke. Med $\alpha=0$ blir resultatet lik vanlig lineær regresjon uten intercept.

```
def f(b1, X, y, la):
    return np.sum((y - X @ b1)**2) + la * np.sum(b1**2)
```

```
def g(X, y, la):
    b0 = y.mean()
    yd = y - b0
    Xd = X - X.mean(axis=0)
    p = Xd.shape[1]
    A = Xd.T @ Xd + la * np.eye(p)
    b1 = np.linalg.solve(A, Xd.T @ yd)
    return b1
```

Forklaring i ord. f beregner summen av kvadrerte residualer pluss Ridge-straff. g finner koeffisientene som minimiserer dette for gitt α . Intercept håndteres som $\text{mean}(y)$ fordi data er sentrert rundt gjennomsnittet.

Leave-one-out-kryssvalidering for optimal α . Tren modellen n ganger, hver gang med én observasjon holdt utenfor. Merk: prediksjonen må bruke samme sentrerte skala som g .

```
def loo(la, X, y):
    n = len(y)
    b0 = y.mean()
    yd = y - b0
    Xd = X - X.mean(axis=0)
    ydpred = np.zeros(n)
    for i in range(n):
```

```
        idx = np.arange(n) != i
        b1 = g(X[idx], y[idx], la)
        ydpred[i] = Xd[i] @ b1
    return np.mean((yd - ydpred)**2)
```

```
lavec = np.linspace(4500, 5000, 10)
testMSE = np.array([loo(la, X, y) for la in lavec])
best_la = lavec[np.argmin(testMSE)]
```

6. Bootstrap, eksamen 2025

Bootstrap. Trekk n observasjoner med tilbakelegging, beregn statistikk, gjenta B ganger. Resultatet er en empirisk samplingsfordeling.

```
np.random.seed(42)
n = len(y)
B = 10000
vboot = np.zeros(B)
```

```
for b in range(B):
    yboot = np.random.choice(y, size=n, replace=True)
    vboot[b] = np.var(yboot, ddof=1)
```

```
plt.hist(vboot, bins=40)
ci = np.percentile(vboot, [2.5, 97.5])
```

Bruk $\text{ddof}=1$ for stikkprøvevarians: deler på $n-1$.

7. K-nearest neighbours og lokal lineær regresjon, eksamen 2024

Vanlig K-nearest neighbours. Prediksjonen er gjennomsnittet av y for de K nærmeste observasjonene.

```
def knn(x0, x, y, K=3):
    d = np.abs(x - x0)
    o = np.argsort(d)[:K]
    return np.mean(y[o])
```

2024-funksjonen er K-nearest-neighbours-lignende, men lokal lineær regresjon. Den velger de K nærmeste observasjonene og fitter $y_1 \sim x_1$, i stedet for å ta gjennomsnittet.

```
def local_lm(x0, x, y, K=3):
    d = np.abs(x - x0)
    o = np.argsort(d)[:K]
    x1 = x[o]
    y1 = y[o]
    X1 = sm.add_constant(x1)
    reg = sm.OLS(y1, X1).fit()
    return reg.predict([1, x0])[0]
```

Leave-one-out-kryssvalidering for K .

```
def loo_K(K, x, y):
    n = len(x)
    ypred = np.zeros(n)
    for i in range(n):
        idx = np.arange(n) != i
        ypred[i] = local_lm(x[i], x[idx], y[idx], K)
    return np.mean((y - ypred)**2)
```

```
Ks = range(2, 11) # lokal lm trenger minst K=2
mse = np.array([loo_K(K, x, y) for K in Ks])
best_K = list(Ks)[np.argmin(mse)]
```

Flere prediktorer. Bytt avstand til euklidisk avstand, helst etter standardisering:

```
d = np.sqrt(np.sum((X - x0)**2, axis=1))
```

8. Logistisk regresjon og terskel

Modell. $P(Y=1|X) = 1/(1+\exp(-(b_0 + bX)))$. Estimeres med maximum likelihood. Koeffisientene er lineære i log-odds.

Forelesers statsmodels-variant.

```
X_train = sm.add_constant(train[["Lag2"]])
y_train = (train["Direction"] == "Up").astype(int)
m = sm.GLM(y_train, X_train, family=sm.families.Binomial()).fit()
```

```
X_test = sm.add_constant(test[["Lag2"]])
prob = m.predict(X_test)
pred = (prob > 0.5).astype(int)
```

Formelvariant.

```
df["y"] = (df["Direction"] == "Up").astype(int)
m = smf.logit("y ~ Volume + Lag1 + Lag2 + Lag3 + Lag4 + Lag5",
              data=train).fit()
prob = m.predict(test)
```

Terskel ved ubalanserte klasser. Finn terskel på treningsdata, evaluer på testdata.

```
prob_tr = m.predict(train)
for c in [0.01, 0.02, 0.05, 0.10, 0.50]:
    pred_tr = prob_tr > c
    print(c, pd.crosstab(train["y"], pred_tr, normalize="index"))
```

```
prob_te = m.predict(test)
pred_te = prob_te > chosen_cutoff
pd.crosstab(test["y"], pred_te, normalize="index")
```

9. Trær, random forest og boosting

Decision tree. Regresjon: bladprediksjon er gjennomsnitt. Klassifikasjon: majoritetsklasse/sannsynlighet. Begrens treet med `max_depth` og `min_samples_leaf`.

```
tree = DecisionTreeRegressor(min_samples_leaf=15, max_depth=3,
                             random_state=0)
tree.fit(X_train, y_train)
plot_tree(tree, feature_names=X_train.columns, filled=False, fontsize=8)
pred = tree.predict(X_test)
```

Pruning.

```
path = tree.cost_complexity_pruning_path(X_train, y_train)
grid = GridSearchCV(DecisionTreeRegressor(random_state=0),
                    {"ccp_alpha": path.ccp_alphas}, cv=5)
grid.fit(X_train, y_train)
best_tree = grid.best_estimator_
```

Random forest og bagging, forelesers mønster.

```
bag = RandomForestRegressor(n_estimators=500,
                           max_features=X_train.shape[1],
                           random_state=123) # bagging
bag.fit(X_train, y_train)
```

```
rf = RandomForestRegressor(n_estimators=500, max_features=6,
                          random_state=123)
rf.fit(X_train, y_train)
pred_rf = rf.predict(X_test)
```

```
pd.Series(rf.feature_importances_,
          index=X_train.columns).sort_values().plot.barh()
```

Klassifikasjon.

```
rfc = RandomForestClassifier(n_estimators=500, max_features="sqrt",
                             random_state=123)
rfc.fit(X_train, y_train)
prob = rfc.predict_proba(X_test)[: , 1]
pred = prob > cutoff
```

Boosting.

```
boost = GradientBoostingRegressor(n_estimators=1000,
                                  learning_rate=0.01,
                                  max_depth=2,
                                  random_state=123)
```

```
boost.fit(X_train, y_train)
```

For binær respons:

```
gbc = GradientBoostingClassifier(n_estimators=1000,
                                 learning_rate=0.01,
                                 max_depth=2,
                                 random_state=123)
```

```
gbc.fit(X_train, y_train)
prob = gbc.predict_proba(X_test)[: , 1]
```

10. Generalized additive models, splines og backfitting

Generalized additive model. $y = b_0 + f_1(x_1) + f_2(x_2) + \dots + e$. Godt når noen prediktorer har ikke-lineær sammenheng med y .

Finn kandidater til ikke-linearitet.

```
for i, col in enumerate(var_names):
    xi = X[:, i]
    r2_lin = sm.OLS(y, sm.add_constant(xi)).fit().rsquared
    Xq = sm.add_constant(np.column_stack([xi, xi**2]))
    r2_quad = sm.OLS(y, Xq).fit().rsquared
    print(col, round(r2_quad - r2_lin, 3))
```

B-spline brukt i en generalized additive model i Python.

```
bs = BSpline(internal_knots=[-1, 0, 1], degree=3, intercept=False)
X3_sp = bs.fit_transform(X[:, 2].reshape(-1, 1))
X_gam = np.column_stack([np.ones(len(y)), X[:, 0], X[:, 1], X3_sp, X[:,
    ↪ 3]])
gam = sm.OLS(y, X_gam).fit()
```

```
mse_gam = np.mean((y - gam.predict(X_gam))**2)
mse_ols = np.mean((y - ols.predict(X_ols))**2)
mse_all = np.mean((y - y.mean())**2)
R2_gam = 1 - mse_gam / mse_all
R2_ols = 1 - mse_ols / mse_all
```

Kubisk spline fra scratch.

```
def h(x, xi):
    return np.maximum((x - xi)**3, 0)
```

```
X_sp = np.column_stack([np.ones(n), x, x**2, x**3,
                        h(x, 3.0), h(x, 6.0)])
fit = sm.OLS(y, X_sp).fit()
```

Backfitting-forklaring. Start med $b_0 = \text{mean}(y)$. Estimer f_1 på residualen fra de andre funksjonene, demean residualen, estimer f_2 , og gjenta til funksjonene nesten ikke endrer seg.

```
def knn(x0, x, y, K=20):
    d = np.abs(x - x0)
    return np.mean(y[np.argsort(d)[:K]])
```

```
b0 = y.mean()
yd = y - b0
f1 = b0 + np.array([knn(x0, x1, yd, 20) for x0 in x1])
res = y - f1
res = res - res.mean()
f2 = b0 + np.array([knn(x0, x2, res, 20) for x0 in x2])
```

11. Polynom og step functions

Polynom. Grad K gir basis $1, x, x^2, \dots, x^K$.

```
Xpoly = np.vander(age, K + 1, increasing=True)
fit = sm.OLS(wage, Xpoly).fit()
```

Velg grad med validation set.

```
mse = []
for k in range(1, 11):
    Xtr = np.vander(age_train, k + 1, increasing=True)
    fit = sm.OLS(wage_train, Xtr).fit()
    Xte = np.vander(age_test, k + 1, increasing=True)
    mse.append(np.mean((wage_test - fit.predict(Xte))**2))
best_k = np.argmin(mse) + 1
```

Step function.

```
train = train.assign(age_cut=pd.cut(train["age"], 4))
Xtr = pd.get_dummies(train["age_cut"], drop_first=True).astype(float)
Xtr = sm.add_constant(Xtr)
```

12. Support vector machines og nevrale nettverk

Dette er mindre dominerende i 2024/2025-eksamenene, men er i pensum.

Support vector machine. I sklearn er C invers regulering: liten C gir sterkere regulering og mykere/bredere margin; stor C gir mer fleksibel grense. Radial basis function-kernel, skrevet `rbf` i sklearn, bruker γ : stor γ gir mer lokal/kompleks grense.

```
param_rbf = {"C": [0.01, 1, 100, 1000], "gamma": [0.5, 1, 2, 4]}
tune = GridSearchCV(SVC(kernel="rbf"), param_rbf, cv=5)
tune.fit(X_train, y_train)
pred = tune.best_estimator_.predict(X_test)
```

Nevrale nettverk. Foreleser: kun sklearn `MLPRegressor` og `MLPClassifier`, ikke PyTorch eller mer avanserte nettverkstyper. Standardiser X .

```
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
```

```
nn = MLPRegressor(hidden_layer_sizes=(15,), activation="relu",
                  max_iter=100000, random_state=0)
nn.fit(X_train_s, y_train)
pred = nn.predict(X_test_s)
```

13. Principal component analysis og clustering

Unsupervised learning. Vi observerer bare forklaringsvariabler X , ikke en responsvariabel y . Brukes til eksplorativ analyse og hypoteser.

Principal component analysis. Lager lineære kombinasjoner av X . Første principal component maksimerer varians, andre principal component maksimerer gjenværende varians og er ukorrelert med den første. Standardiser hvis variabler har ulik skala.

```
def summary_pca(fit):
    return pd.DataFrame({
        "SD": np.sqrt(fit.explained_variance_),
        "PropVar": fit.explained_variance_ratio_,
        "CumProp": np.cumsum(fit.explained_variance_ratio_)
    })
```

```
X_s = StandardScaler().fit_transform(X)
pca = PCA()
scores = pca.fit_transform(X_s)
summary_pca(pca)
```

```
loadings = pd.DataFrame(pca.components_.T, index=X.columns,
                        columns=[f"PC{i+1}" for i in
                               range(pca.n_components_)])
```

K-means og hierarkisk clustering.

```
X_s = StandardScaler().fit_transform(X)
```

```
km = KMeans(n_clusters=3, random_state=1, n_init=10)
km.fit(X_s)
pd.crosstab(km.labels_, y_true)
```

```
hc = AgglomerativeClustering(distance_threshold=0, n_clusters=None,
                             linkage="complete")
```

```
hc.fit(X_s)
lm = compute_linkage(hc)
dendrogram(lm, no_labels=True)
clusters = cut_tree(lm, n_clusters=3).reshape(-1)
```

Linkage: `complete` = maks avstand, `single` = min, `average` = snitt, `ward` = minimerer varians.

14. Modellvalg i caseoppgaver

Forklaringscase: "Hvorfor skjer Y?"

1. Deskriptiv analyse: vis hvilke variabler som er assosiert med Y .
2. Logistisk regresjon eller vanlig lineær regresjon for tolkbare koeffisienter.
3. Tre eller random forest for ikke-linearitet og variabelviktighet.
4. Konklusjon: svar med de viktigste variablene, ikke bare "modell X var best".

Prediksjonscase: "Hvem vil få Y i fremtiden?"

1. Fjern lekkasjevariabler og variabler som ikke er kjent på prediksjonstidspunktet.
2. Del i treningsdata og testdata.
3. Velg terskel på treningsdata hvis klassene er ubalanserte.
4. Evaluer på testdata med krysstabell og rad-proporsjoner.
5. Sammenlign logistisk regresjon mot random forest eller boosting med samme variabler.

Typiske formuleringer.

- "Jeg bruker testsettet bare til siste evaluering."
- "Terskelen 0.5 er ikke nødvendigvis passende fordi positiv klasse er sjelden."
- "Random forest forbedrer prediksjon, men logistisk regresjon er lettere å tolke."
- "Variabelviktighet viser prediktiv betydning, ikke nødvendigvis kausal effekt."